# Reconocimiento Automatizado De Emociones Faciales en Niños con Trastorno del Espectro Autista mediante Técnicas De Machine Learning y Computer Vision

Seleme Rocío

IES21

5° Semestre - Procesamiento de Imágenes

Abril 2025

Córdoba, Argentina

# ÍNDICE

INTRODUCCIÓN..... 3

OBJETIVOS..... 5

MARCO TEÓRICO..... 6

1.1 Redes Neuronales Convolucionales (CNN).....6

1.2 Arquitecturas aplicadas en el proyecto..... 8

1.3 Hiper Parámetros..... 9

METODOLOGÍA.....13

1.1 Experimentos..... 13

1.2 Resultados y análisis..... 18

CONCLUSIÓN..... 27

REFERENCIAS.....30

# INTRODUCCIÓN

El reconocimiento de expresiones faciales representa un desafío significativo para niños con Trastorno del Espectro Autista (TEA), quienes manifiestan dificultades para interpretar, percibir y expresar señales como gestos, miradas y emociones (Rodríguez, 2021). Estas limitaciones afectan su capacidad para integrarse socialmente, como así también, participar tanto en entornos escolares como familiares (G.Lorenzo-Lledó, A.Lorenzo-Lledó, Quevedo, 2024). Según la Organización Mundial de la Salud (2023), se estima que 1 de cada 100 niños en el mundo presenta esta condición, la cual se manifiesta con distintos grados de intensidad y para cada persona de manera diferente (Rossi, 2007).

En este contexto, la inteligencia artificial (IA) aplicada al reconocimiento de emociones se presenta como una herramienta prometedora. En particular, las redes neuronales convolucionales (CNN) han demostrado ser eficaces en la clasificación de expresiones faciales en tiempo real, incluso en entornos educativos y terapéuticos (Salhi et al., 2022). Esta tecnología puede integrarse en sistemas interactivos, como robots sociales, capaces de captar emociones básicas (como alegría, tristeza o enfado) y retroalimentar la interacción con el usuario.

Diversas investigaciones recientes han explorado el uso del robot humanoide NAO en actividades con niños con TEA, mostrando que la combinación de inteligencia artificial y robótica social mejora la participación, comunicación y el reconocimiento de las emociones en estos niños (Lorenzo-Lledó et al., 2024). Esta tecnología no busca reemplazar la interacción entre las personas, sino actuar como un mediador emocional y social, facilitando la comprensión del estado de ánimo en ambientes estructurados y seguros.

Las emociones son respuestas internas subjetivas que surgen a partir de nuestros pensamientos, y son fundamentales en la manera en que nos vinculamos con el entorno (Rodríguez, 2021). En el caso de los niños con Trastorno del Espectro Autista, el

reconocimiento y la expresión emocional presentan limitaciones significativas, dificultando el desarrollo de habilidades sociales y la comprensión de las señales afectivas durante las interacciones (Rodríguez, 2021). Estas dificultades afectan no solo la calidad del vínculo con su entorno familiar y escolar, sino también su bienestar emocional y comunicativo.

Si bien el TEA no es una enfermedad curable, sino una condición del neurodesarrollo, existen alternativas para ayudar al niño a mejorar su calidad de vida e integración social (Salhi et al., 2022). Una de las estrategias más prometedoras en los últimos años es el uso de tecnología basada en visión por computadora e inteligencia artificial, en particular a través del reconocimiento automático de expresiones faciales.

Distintos estudios recientes demuestran que los sistemas tecnológicos aplicados a contextos terapéuticos pueden mejorar conductas como la imitación, la atención compartida o la expresión emocional. Por ejemplo, Salhi et al. (2022) demostraron que los niños con TEA que interactuaron con robots mostraron un incremento significativo en sus comportamientos sociales. En un estudio clínico piloto, Conti et al. (2015) evaluaron el uso del robot NAO con tres niños con TEA y discapacidad intelectual, observando avances positivos en la imitación corporal. Este tipo de dispositivos no solo se integran fácilmente en contextos terapéuticos, sino que también funcionan como mediadores entre el niño y su entorno.

El desarrollo de herramientas automatizadas de reconocimiento de emociones puede, además, reducir múltiples barreras estructurales como la escasez de profesionales especializados (principalmente en zonas no urbanas), los altos costos de tratamiento o la fatiga generada por la repetición de tareas en terapias tradicionales (Holeva et al., 2022). En este sentido, los modelos basados en redes neuronales convolucionales se presentan como una solución accesible, escalable y adaptable a distintos contextos.

Por lo tanto, implementar un sistema de análisis emocional facial mediante IA representa un aporte innovador y relevante al campo de la intervención educativa y

terapéutica en niños con TEA, facilitando su inclusión y promoviendo una comunicación más fluida con su entorno.

En función de esta problemática, el presente trabajo propone desarrollar un modelo de clasificación de emociones faciales mediante redes neuronales convolucionales, con el objetivo de aportar una solución tecnológica que facilite la comunicación con niños con TEA. Se plantea como una herramienta de apoyo para terapeutas, docentes y familiares, que permita interpretar emociones de manera automática a partir del análisis visual de expresiones faciales, promoviendo así un aprendizaje e interacción más fluida y eficiente con niños que presentan esta condición.

## OBJETIVOS

*Objetivo General*

Desarrollar un prototipo de reconocimiento de emociones faciales utilizando una red neuronal de convolución (CNN) para ayudar en la interpretación emocional de los niños con Trastorno del Espectro Autista para junio del 2025.

*Objetivos Específicos:*

- Desarrollar una CNN base para determinar un benchmark con 2 Capas Convolucionales y 1 capa dense.
- Desarrollar un modelo de red neuronal convolucional basado en la arquitectura Mini-Xception para la detección automática de siete emociones básicas en imágenes faciales.
- Optimizar los modelos en función de los hiper parámetros (Learning Rate, Épocas, Dropout y Optimizador)
- Alcanzar un Accuracy superior al 55% en el conjunto de validación.

# MARCO TEÓRICO

## 1.1 Redes Neuronales Convolucionales (CNN)

Dentro del campo del Machine Learning (ML), se encuentra el Deep Learning (DL) o aprendizaje profundo. Según Howard (2019), esta técnica se basa en redes neuronales artificiales capaces de procesar grandes volúmenes de datos y extraer patrones complejos. Uno de los modelos más utilizados en este ámbito son las Redes Neuronales Convolucionales (CNN), particularmente eficaces en tareas de reconocimiento, clasificación y segmentación de imágenes.

Las CNN son una de las redes neuronales profundas más populares y ampliamente empleadas. Alzubaidi et al (2021) identifican tres ventajas clave de este tipo de neuronas: representaciones equivalentes, interacciones dispersas y compartición de parámetros. A diferencia de las redes completamente conectadas (FC) convencionales, las CNN utilizan pesos compartidos y conexiones locales, favoreciendo la reducción de parámetros entrenables, y por lo tanto, contribuye a la red a mejorar la generalización y disminuir el sobreajuste. Asimismo, el aprendizaje simultáneo de las capas de extracción de características y la capa de clasificación, genera salidas del modelo organizadas, fuertemente vinculadas con las características extraídas. Por estas razones, implementar redes neuronales profundas a gran escala resulta considerablemente más sencillo con CNN que con otros tipos de redes (Alzubaidi et al., 2021).

Un tipo común de CNN consta de numerosas capas de convolución que preceden a las capas de agrupación, mientras que las capas finales son capas fully connected (FC) o totalmente conectadas (Alzubaidi et al., 2021). La siguiente figura (1) ilustra un ejemplo de arquitectura de una CNN para la clasificación de imágenes:

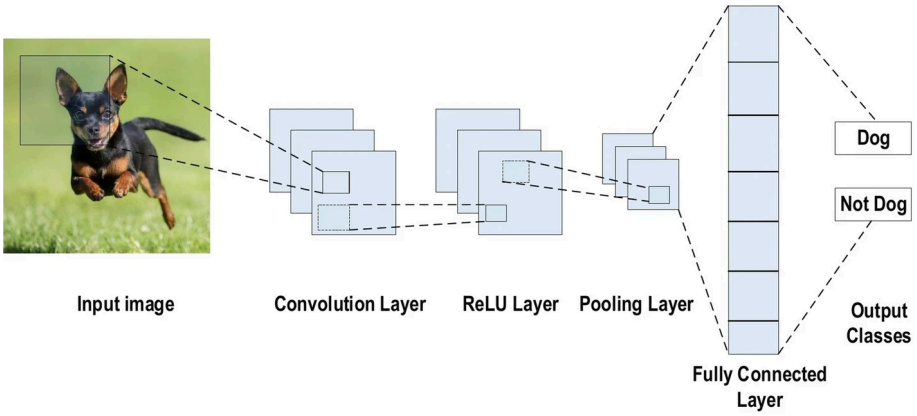


Figura (1)

En la arquitectura de una CNN, una de las capas más importante es la 'Capa convolucional'. Esta consta de un conjunto de filtros convolucionales denominados núcleos, donde la imagen de entrada, expresada como métricas N-dimensionales, se convoluciona con estos filtros para generar el mapa de características de salida. En otras palabras, extrae mapas de características aplicando múltiples kernels que actúan como detectores de patrones locales (Alzubaidi et al., 2021).

Como segunda capa tenemos la 'Capa de Agrupación' o Pooling, que consiste en reducir los mapas de características de gran tamaño para crear mapas de características más pequeños. Además, conserva la mayor parte de la información o característica dominante en cada paso de la etapa de agrupación. Existen varios tipos de métodos de agrupación, como ser Max Pooling (agrupación máxima) y Average Pooling (agrupación promedio).

Por otro lado, se encuentra la capa 'Función de activación' que introduce no linealidad y mitiga el desvanecimiento del gradiente. Lo que hace es mapear la entrada con la salida, donde el valor de entrada se determina calculando la suma ponderada de la entrada de la neurona junto con su sesgo (si lo hay). Esto significa, que la función de activación decide si se activa o no una neurona con referencia a una entrada específica, creando la salida correspondiente (Alzubaidi et al., 2021). Este rendimiento no lineal de las capas de activación implica que la asignación de entrada a salida será no lineal, permitiendo a la CNN aprender

funciones extremadamente complejas. Entre los tipos de funciones de activación más utilizados en las CNN están la ReLU, Sigmoide, Tanh, Softmax, entre otras.

Por último, tenemos la 'Capa Completamente Conectada', que normalmente se ubica al final de cada arquitectura y dentro de esta capa, cada neurona está conectada a todas las neuronas de la capa anterior, lo que se conoce como el enfoque Completamente Conectado (FC- Fully Connected). En simples palabras, se utiliza como clasificador CNN, donde la entrada de la capa FC proviene de la última capa de agrupamiento o de convolución. Esta entrada se presenta en forma de vector, que se crea a partir de los mapas de características tras el aplanamiento. La salida de la capa FC representa la salida final de la red neuronal, lo cual refleja las probabilidades de cada clase (Alzubaidi et al., 2021).

Ahora bien, a pesar de los beneficios que tienen las CNN, también presentan ciertas desventajas importantes. Principalmente, como plantean Alzubaidi et al. (2021), se requieren grandes volúmenes de datos para poder entrenar estos modelos de alto rendimiento, lo cual puede ser un gran limitante en escenarios donde la recopilación de datos es escasa. Además, su entrenamiento es computacionalmente intensivo, demandando recursos considerables. Por último, el rendimiento de las CNN es altamente sensible a la elección y el ajuste de hiper parámetros, lo que puede complicar su configuración y optimización.

## 1.2 Arquitecturas aplicadas en el proyecto

Expresan Alzubaidi et al. (2021) que la arquitectura del modelo es un factor clave para mejorar el rendimiento de diferentes aplicaciones. A lo largo de los años, se han logrado varias modificaciones en la arquitectura de una CNN, que incluyen reformulación estructural, regularización, optimizaciones de parámetros, etc. En particular, los cambios más importantes en arquitecturas se realizaron en el uso de la profundidad de la red. En este apartado,

revisamos las arquitecturas de CNN empleadas para el proyecto de Reconocimiento Automatizado De Emociones Faciales, comenzando por el modelo 'Mini-Xception'.

De acuerdo con Fatima et al. (2021), el modelo Mini-Xception se inspira en el algoritmo Xception, el cual consiste en un modelo de reconocimiento mejorado que utiliza redes convolucionales residuales para el reconocimiento de emociones. Esta arquitectura combina módulos residuales y convoluciones separables en profundidad. Los módulos residuales modifican la función de mapeo deseada entre dos capas sucesivas para que las características aprendidas sean la diferencia entre el mapa de características original y las características deseadas. En consecuencia, las características deseadas H(x) se modifican para resolver un problema de aprendizaje más sencillo, F(X), de modo que: $H(x) = F(x) + x$.

En el modelo propuesto, los parámetros de las capas convolucionales se reducen mediante convoluciones separables en profundidad. Estas convoluciones separables se componen de dos capas diferentes: convoluciones en profundidad y convoluciones puntuales. El objetivo principal de estas capas es separar las correlaciones cruzadas espaciales de las correlaciones cruzadas de canal. Para ello, primero se aplica un filtro $D \times D$ a cada M canales de entrada y, a continuación, se aplican N filtros de convolución $1 \times 1 \times M$ para combinar los M canales de entrada en N canales de salida. La aplicación de convoluciones $1 \times 1 \times M$ combina cada valor del mapa de características sin considerar su relación espacial dentro del canal. (Fatima et al., 2021). De esta manera, las convoluciones separables en profundidad reducen el costo computacional.

## 1.3 Hiper Parámetros

La elección de hiperparámetros influye considerablemente en el rendimiento de las CNN. Zhao et al. (2024) establecen que la selección de hiperparámetros óptimos requiere de gran conocimiento, ya que cualquier pequeña alteración puede tener una influencia

significativa en los resultados finales del entrenamiento. En este contexto, se pueden utilizar algoritmos metaheurísticos para ajustar automáticamente los hiperparámetros mediante búsquedas aleatorias y dirigidas basadas en hallazgos previos. Para el desarrollo de este proyecto, se consideraron y ajustaron varios hiperparámetros clave con el fin de optimizar el entrenamiento y el rendimiento del modelo de clasificación de emociones faciales. A continuación, se detallan los hiperparámetros más relevantes empleados.

En primer lugar, hablamos del Learning Rate (η) o tasa de aprendizaje, el cual representa el paso de aprendizaje para cada optimización y es la función objetivo. Según Zhao et al. (2024) es un parámetro importante del descenso de gradiente ya que determina cuánto se debe avanzar en la función objetivo para encontrar el punto mínimo. Existen dos extremos que pueden ocurrir con la configuración del paso de aprendizaje, primero si el paso de aprendizaje es demasiado pequeño, se necesitarán muchas iteraciones antes de que el algoritmo converja, lo cual consume mucho tiempo. Por otro lado, si el paso de aprendizaje es demasiado grande, el punto mínimo se omitirá o incluso podría no encontrarse (podría divergir). Por ello, es necesario seleccionar la tasa de aprendizaje con cuidado para que no influya negativamente en el proceso de aprendizaje.

Como segundo parámetro, tenemos el batch normalization (BN) o normalización por lotes, el cual consiste en una técnica de regularización de redes neuronales que garantiza el rendimiento de las activaciones de salida. Este rendimiento sigue una distribución gaussiana unitaria, que al restar la media y dividir por la desviación estándar, normalizará la salida en cada capa. Con lo cual, este método unifica la distribución de los valores del mapa de características estableciéndolos en media cero y varianza unitaria. Además, la capa BN ayuda a mitigar el problema de la desaparición y la explosión de gradiente, mejora la adaptabilidad de la red a diferentes datos de entrada, facilita la inicialización de pesos, acelera el proceso de entrenamiento de la red neuronal y mejora su generalización. Asimismo,

podemos agregar que si estamos en presencia de lotes grandes se obtendrá gradientes estables, mientras que lotes más pequeños favorecen la generalización de la red (Zhao et al., 2024).

Otro posible parámetro a tener en cuenta son las épocas de entrenamiento o Epochs, las cuales representan una repetición completa de la actualización de un parámetro que involucra todo el conjunto de datos de entrenamiento de una sola vez. Si contamos con demasiadas épocas es posible caer en un sobreajuste, mientras que si empleamos unas pocas el modelo tenderá a un subajuste (Alzubaidi et al., 2021).

Un cuarto factor a considerar es Adaptive Moment Estimation (Adam) o estimación de momento adaptativo. Considerando lo planteado por Alzubaidi et al. (2021), el parámetro Adam es otra técnica de optimización o algoritmo de aprendizaje muy utilizado y se representa mediante la matriz Hessiana, que emplea una derivada de segundo orden. Dos de sus ventajas son una mayor eficiencia de memoria y un menor consumo de potencia computacional. El mecanismo de Adam consiste en calcular la Learning Rate, o tasa de aprendizaje, adaptativa para cada parámetro del modelo. De esta manera, Adam es un algoritmo de optimización adaptativo que calcula una tasa de aprendizaje diferente para cada peso y sesgo, o parámetro, de la red neuronal; y esta tasa de aprendizaje individual se ajusta dinámicamente durante el proceso de entrenamiento.

Por último, tenemos la técnica Dropout ampliamente utilizada para la generalización. El dropout consiste en que, durante cada época de entrenamiento, se irán descartando neuronas aleatoriamente con una probabilidad predeterminada. De esta forma, la capacidad de selección de características se distribuye equitativamente entre todo el grupo de neuronas, obligando al modelo a aprender diferentes características independientes y mejorando la generalización. Por lo tanto, durante el proceso de entrenamiento, la neurona descartada no formará parte de la propagación hacia atrás ni hacia adelante. En cambio, la red a escala real

se utiliza para realizar predicciones durante el proceso de prueba (Alzubaidi et al., 2021). En la siguiente imágen Figura(2), observamos cómo se distribuyen las conexiones de las neuronas en capas con Dropout:

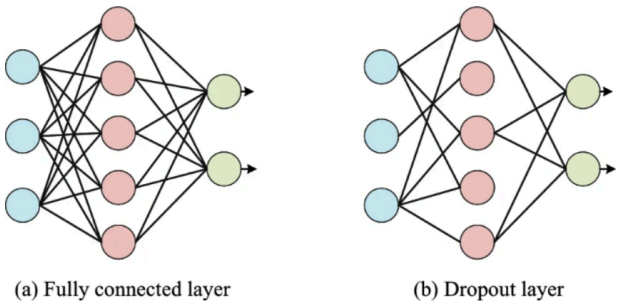


Figura (2)

Más allá de la configuración de los hiperparámetros, la capacidad de un modelo de CNN para generalizar a datos no vistos y evitar el sobreajuste depende en gran medida de la diversidad y el tamaño del conjunto de datos de entrenamiento con el que se cuente. Por ello, para lograr contar con una cantidad considerable de datos se emplean técnicas de Data Augmentation o aumento de datos. Explican Alzubaidi et al. (2021), que el Data Augmentation consiste en ampliar artificialmente el tamaño del conjunto de entrenamiento mediante la aplicación de diversas transformaciones, lo que ayuda a aumentar la robustez del modelo y su capacidad de reconocimiento en diferentes condiciones. A continuación, las soluciones alternativas de aumento de datos que se aplican en este proyecto son: Flipping o Inversión, que consiste en invertir el eje vertical u horizontal; y Rotation o rotación que busca rotar la imágen a la izquierda o a la derecha entre 0 y 360 grados alrededor del eje.

# METODOLOGÍA

## 1.1 Experimentos

Con el fin de validar los conceptos teóricos y llevar a la práctica el diseño del modelo de reconocimiento de emociones faciales, se procedió a la implementación y experimentación de diferentes pruebas de codificación. En esta sección, se detallarán los experimentos realizados, incluyendo la aplicación de técnicas de preprocesamiento como el aumento de datos, el entrenamiento del modelo y los ajustes de hiperparámetros, con el objetivo de evaluar el rendimiento y la robustez de las arquitecturas propuestas.

**Experimento n°1**: en este primer experimento se procedió a levantar el conjunto de imágenes descargado de Kaggle, visualizar el dataset y poder tener así una idea general sobre el conjunto de datos que estamos trabajando. Luego se crea una función de preprocesamiento, donde vamos a convertir las imágenes de BGR a RGB, las normalizamos, dimensionamos todas a 48x48, mapeamos las etiquetas de las emociones a números, creamos listas para almacenar los valores de X e y, realizamos shuffle del conjunto de datos y luego hacemos la división correspondiente para entrenamiento. Después realizamos data augmentation, para poder contar con más imágenes para entrenar. Para ello, se aplicaron transformaciones de rotación (+10° / -10°) y flip horizontal. Además, se realiza la división en validación, de los datos aumentados, para entrenar con ellos y probar diferentes hiperparametros.

En la creación de modelos, para establecer un benchmark inicial, se diseñó una CNN sencilla 'modelo_base_1' compuesta por dos bloques convolucionales principales: el primero con 32 filtros y el segundo con 64 filtros, ambos utilizando un tamaño de kernel de 3x3. Cada bloque incluye Batch Normalization para estabilizar el entrenamiento, activación ReLU para brindar no linealidad, MaxPooling2D para reducción de dimensionalidad y Dropout (0.25)

como regularización. Tras estos bloques, las características extraídas se aplanan (Flatten()) y se pasan a una capa Dense oculta de 128 neuronas con ReLU y Dropout (0.5), finalizando con una capa de salida Dense de 7 neuronas con activación softmax. El modelo se compiló utilizando el optimizador Adam, la función de pérdida sparse_categorical_crossentropy y la métrica de accuracy.

Por otro lado, se desarrolla una alternativa más eficiente, cuya arquitectura está en Mini-Xception. La red 'model' inicia con una capa convolucional estándar, seguida de múltiples bloques que incorporan SeparableConv2D con un número de filtros (16, 32, 64). Cada uno de estos bloques incluye BatchNormalization, activación ReLU, MaxPooling2D para la reducción de dimensionalidad y Dropout (0.25) para regularización. La fase final de clasificación utiliza GlobalAveragePooling2D para consolidar las características, seguida de una capa Dense oculta de 64 neuronas con ReLU y Dropout (0.5), y una capa de salida Dense con softmax para clasificar las 7 emociones. El modelo se compila con el optimizador Adam, la función de pérdida sparse_categorical_crossentropy y la métrica accuracy.

**Experimento n°2**: Como segunda prueba, se propone usar las imágenes en escalas grises. Si bien no está comprobado que el color influye notablemente en la detección de las emociones o expresiones del rostro, pero lo aplicamos para evaluar si se dan cambios notorios. Asimismo, se decidió redimensionar las imágenes en un tamaño superior a 48x48, como ser 64x64 / 96x96, para ver si esto ayuda a mejorar el rendimiento del modelo ya que las imágenes se mostraban con una pequeña distorsión por la redimensión inicial.

Por otra parte, se probará de usar una menor regularización con dropout, durante el entrenamiento, ya que una probabilidad del 0.5 podría ser un poco alta. Esto significa que se están apagando demasiadas neuronas durante el entrenamiento (el 50% en cada paso). Por ende, se introduce mucho ruido y fuerza al modelo a ser muy simple en cada iteración.

Posterior a la creación del modelo base, generamos Callbacks para entrenamiento. Esto es conveniente para parar temprano el entrenamiento cuando no hay progreso en el modelo (Early Stopping) y para reducir el learning rate si la validación no mejora (ReduceLROnPlateau). En resumen, se va a eliminar el Dropout(0.5) de la capa final de Mini-Xception (manteniendo los Dropout(0.25) intermedios), se reduce el learning rate inicial a 1e-4 y se agregan callbacks EarlyStopping y ReduceLROnPlateau para controlar la parada temprana y ajustar el learning rate durante la ejecución. En relación al 'modelo_base_1', se opta por aplicar una regularización más balanceada entre todas las capas, con un dropout de 0.20 para cada una.

**Experimento n°3**: Para el tercer experimento, se explora la capacidad de los modelos pre-entrenados mediante la aplicación de VGG-16, el cual fue inicialmente entrenado con el conjunto de datos ImageNet. Esta arquitectura se basa en el aprendizaje por transferencia, aprovechando la capacidad de VGG-16 para extraer características robustas de imágenes. De esta manera, podremos comparar qué resultados se obtienen y sacar conclusiones más valiosas sobre otros modelos.

Alzubaidi et al. (2021) exponen que el Grupo de geometría visual (VGG) es un modelo multicapa, caracterizado por su gran profundidad permitiendo a la red aprender representaciones más complejas y jerárquicas de las características de una imagen. Esta arquitectura se centra en el uso de filtros pequeños de tamaño 3×3 apilados en múltiples capas. A diferencia de otras estructuras que emplearon filtros grandes, como por ejemplo 11×11 en ZefNet o 7×7 en AlexNet, VGG demostró que apilar varias convoluciones 3×3 secuencialmente equivale a un filtro de tamaño mayor (por ejemplo, dos capas de 3×3 simulan un 5×5), pero con menos parámetros y más funciones de activación intermedias, lo que mejora la capacidad de aprendizaje de la red.

Con respecto al ajuste de la red, se inserta una capa de agrupación máxima después de la capa convolucional, mientras que se implementa el relleno para mantener la resolución espacial. En general, VGG obtuvo resultados significativos para problemas de localización y clasificación de imágenes. Sin embargo, el principal inconveniente de este modelo es su elevado costo computacional ya que la red contiene aproximadamente 140 millones de parámetros, demandando recursos significativos de memoria y cómputo. A continuación, visualizamos en la siguiente figura (3) un ejemplo de la arquitectura VGG de una CNN para la clasificación de imágenes:

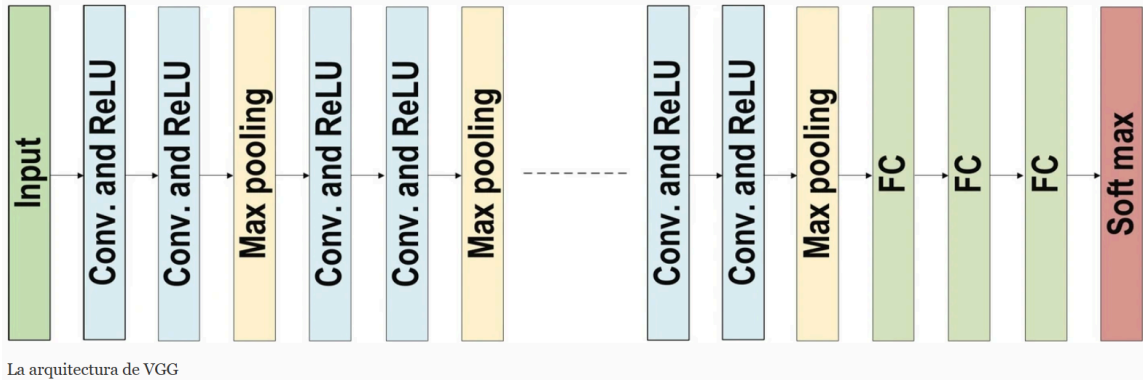


Figura (3)

Dado que el modelo VGG-16 fue pre-entrenado con el conjunto de datos ImageNet, las imágenes de entrada para nuestro modelo deben ser preparadas de acuerdo con sus requerimientos. Si bien nuestras imágenes ya cuentan con un tamaño de 48x48 píxeles y tres canales de color (RGB), solo es necesario realizar dos pasos más de preprocesamiento: primero reescalar los pixeles al rango original de [0,255], ya que las imágenes se las normalizó en el rango [0,1]; y segundo aplicar el preprocesado específico de VGG-16 (BGR centering). Esta función realiza transformaciones como la conversión del orden de los canales de RGB a BGR y la resta de los valores medios de píxeles de ImageNet, de esta manera aseguramos que las nuevas entradas sean consistentes con los datos con los que VGG-16 fue originalmente entrenado.

Una vez preparados los datos, se procedió a construir el modelo VGG-16 adaptado, reutilizando la base convolucional del modelo pre-entrenado y añadiendo nuevas capas para la tarea específica de clasificación de emociones. El modelo cuenta con la siguiente arquitectura: se cargó la parte convolucional de VGG-16 pre-entrenada con pesos de ImageNet (weights='imagenet') y se especificó que no se incluyeran las capas superiores (include_top=False), ya que estas corresponden a la clasificación original de ImageNet. Es necesario que todas las capas de esta base convolucional sean congeladas (base.trainable = False) para evitar que sus pesos, ya muy bien aprendidos, se modifiquen durante el entrenamiento con nuestros datos, lo que podría corromper las características genéricas que ha aprendido. Con respecto a la salida de esta base convolucional congelada, se añadió un nuevo clasificador personalizado, diseñado para nuestro problema de las 7 clases de emociones. Esta cabeza se compone de una capa GlobalAveragePooling2D para reducir las dimensiones espaciales de los mapas de características a un solo vector por mapa, reduciendo significativamente el número de parámetros; una capa Dense de 256 neuronas con activación ReLU diseñada para aprender combinaciones más complejas de las características extraídas y una capa Dropout (con una tasa de 0.3) aplicada después de la capa densa, para regularizar el modelo y prevenir el sobreajuste. Con respecto a la capa de salida tenemos una capa Dense final de 7 neuronas (una por cada clase de emoción), con activación softmax, para generar las probabilidades de pertenencia a cada clase. Por último, el modelo 'model_vgg_1' se compila utilizando el optimizador Adam con una tasa de aprendizaje reducida (1e-4), la función de pérdida sparse_categorical_crossentropy (adecuada para etiquetas de clase enteras) y la métrica accuracy.

**Experimento n°4**: Este apartado se centra en la optimización de hiperparámetros, donde se utilizan algoritmos para ajustar automáticamente los hiperparámetros mediante

búsquedas aleatorias y dirigidas. Dado que model_vgg_2 ya tiene una estructura definida y un buen rendimiento base, podemos enfocarnos en optimizar sus hiperparámetros clave. Para ello, se implementará un Grid Search "manual" a este modelo, similar a lo que venimos haciendo en el experimento anterior, pero adaptado a los parámetros específicos de model_vgg_2:

- Tasa de aprendizaje para el optimizador Adam: learning_rates = [1e-5, 5e-6, 2e-5].
- Tasa de Dropout en la capa densa: dropout_rates = [0.2, 0.3, 0.4].
- Número de neuronas en la capa Dense de la cabeza: dense_neurons_options = [64, 128, 256].

Para realizar la búsqueda de hiperparámetros es crucial que el modelo, con la base de VGG16 y la cabeza personalizada, se reconstruya desde cero dentro de cada iteración del bucle. Esto asegura que cada combinación de hiperparámetros se pruebe con un modelo 'limpio' y que las capas congeladas/descongeladas se reestablezcan correctamente para cada prueba. Además, se incluyen EarlyStopping y ReduceLROnPlateau para cada entrenamiento dentro del Grid Search. Esto es importante para asegurar que cada combinación de hiperparámetros se evalúe bajo condiciones de convergencia y control de sobreajuste similares.

## 1.2 Resultados y análisis

**Resultados n°1**: En la evaluación de ambos modelos, dentro del conjunto de validación, el 'modelo_base_1' obtuvo un Accuracy de 0.1431 y el 'model' un Accuracy: 0.6186. Del primer modelo, notamos que no subió demasiado el valor de la métrica

val_accuracy, en cierta forma se estancó. Esto podría deberse a la escasa profundidad con la que cuenta la red.

Al tener solo dos capas convolucionales, el 'modelo_base_1' probablemente carece de la capacidad de representación necesaria para extraer las características específicas y sutiles que definen las expresiones faciales de las emociones. Una red tan poco profunda limita su habilidad para aprender patrones complejos y distinguir entre las siete clases de emociones, lo que se traduce en un subajuste (underfitting) severo, donde el modelo apenas logra un rendimiento superior al azar en la clasificación.

Por el contrario, la arquitectura Mini-Xception mostró mejores resultados. Al contar con una mayor profundidad y el uso de convoluciones separables en profundidad, demuestra una mejor capacidad para capturar estas características complejas, resultando en un rendimiento considerablemente mejor.

Si bien el modelo de Mini-Xception dio mejores valores, en ambos modelos se observa, que durante el entrenamiento, los valores de val_accuracy van de a poco subiendo pero en momentos vuelven a retroceder. Esto podría deberse a que el modelo estaría aprendiendo más al azar. Esto lo verificamos, poniendo a predecir al modelo que obtuvimos como mejor, con imágenes nuevas para la red, donde observamos que no estaría prediciendo correctamente (ya que para una imágen de tristeza, predice alegría). Con lo cual se procede a seguir probando otras arquitecturas y configuraciones de hiperparámetros.

**Resultados n°2**: Si tomamos en comparación los experimentos 1 y 2, obtuvimos que, con respecto al modelo base_1 no hubo mejoras notables en sus métricas al cambiar el dropout. Suponemos que esto se debe a que la red no es lo suficientemente profunda, lo que la mantiene atrapada en valores muy bajos de val_accuracy (aproximadamente 0.1431).

En relación al modelo inicial Xception, con tasa de aprendizaje constante ($1 \times 10^{-3}$ valor por defecto del optimizador Adam) y sin callbacks, el modelo alcanzó picos de validación superiores al 50 %, pero presentaba una curva de aprendizaje irregular y riesgo de sobreajuste. Al haber eliminado la regularización dropout de 0.5 en la capa final e incorporar ReduceLROnPlateau (factor 0.5, paciencia 10, min_lr $1 \times 10^{-6}$) y EarlyStopping, se logró una trayectoria de aprendizaje un poco más estable y un ajuste más fino del learning rate, aunque esto condujo a valores de validación un poco más bajos que los obtenidos originalmente sin estos callbacks. Esto sugiere que, si bien la reducción automática de la tasa de aprendizaje y la parada temprana aportan robustez y evitan el sobreentrenamiento, pueden frenar temporalmente la obtención de nuevas mejoras en métricas si se aplican demasiado pronto o de forma demasiado agresiva. Por ello, en la práctica, es esencial encontrar un balance del learning rate para conservar la capacidad de exploración inicial sin sacrificar la estabilidad final.

Al aplicar los cambios mencionados, el modelo obtuvo picos de validación de hasta 0.60 (curva roja) en figura(4), y un rendimiento medio en validación 'val accuracy' de aproximadamente 0.59. Aunque este valor resulta ligeramente inferior al 0.62 inicial, la nueva configuración nos aporta una mejor estabilidad de la curva de validación. Es importante notar que, si bien el EarlyStopping detuvo el entrenamiento en el punto óptimo de generalización (cuando la validación se estancó), la diferencia entre la curva de entrenamiento (verde) y la de validación (roja) figura(4), indica que el modelo comenzó a sobreajustarse a los datos de entrenamiento, aprendiendo características específicas de estos y no generalizando completamente. El aprendizaje (curva verde) es más fino, al transicionar de un LR inicial agresivo (1e-3) a pasos menores sólo cuando la pérdida de validación se estanca. Por ende es importante destacar que, si bien las técnicas de regularización implementadas (como el Dropout, EarlyStopping y ReduceLROnPlateau) son efectivas para

controlar y mitigar el sobreajuste, las curvas de aprendizaje aún muestran una tendencia al sobreajuste, entre la precisión de entrenamiento y la de validación.

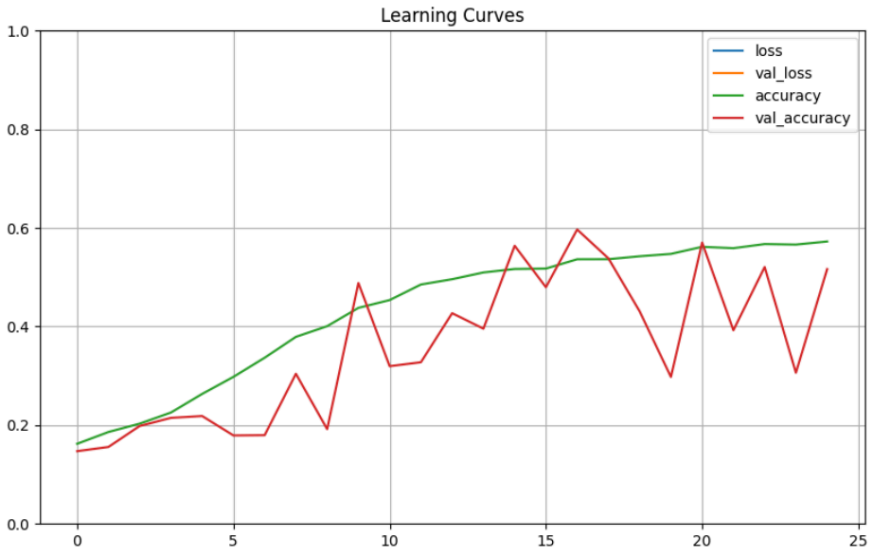


Figura (4)

En relación a los cambios de dimensión y color de las imágenes no se notaron variaciones importantes. Al usar las imágenes en escalas de grises y redimensionarlas a tamaños superiores a 48x48 (como 64x64 o 96x96), no se observó una variación notoria en las métricas evaluadas, lo que deducimos que la información de color y la resolución de 48x48 son adecuadas.

**Resultados n°3**: Tomando en consideración los modelos anteriores y el de arquitectura VGG-16, 'model_vgg_1', podemos decir que se obtuvo un valor de 0.50 en el conjunto de validación. Por otra parte, se observó que el modelo pre entrenado, si bien no obtuvo un accuracy mayor en comparación al Mini-Xception, pero los valores de las métricas durante el entrenamiento no presentaban esos retrocesos constantes como en el resto de los modelos. Observando las curvas de aprendizaje de 'model_vgg_1' figura (5), en una configuración inicial sin callbacks, vemos que el entrenamiento asciende hasta

aproximadamente 0.65 de accuracy y una validación que termina alrededor de 0.48 sin callbacks. Con esto podemos decir que tenemos un modelo base razonable, teniendo en cuenta que Mini-Xception con augmentation y callbacks estuvo en 0.50–0.55. Sin embargo, se observa un sobreajuste notable a partir de la época 12, ya que la curva de accuracy de entrenamiento (verde) continúa mejorando significativamente, mientras que la curva de validación (rojo) se estanca o crece a un ritmo mucho menor, lo que indica que el modelo está comenzando a memorizar los datos de entrenamiento en lugar de generalizar. Si bien congelar la parte convolucional de VGG-16 ayuda a mitigar este fenómeno en las capas de extracción de características, el bloque de clasificación aún muestra señales de sobreajuste.

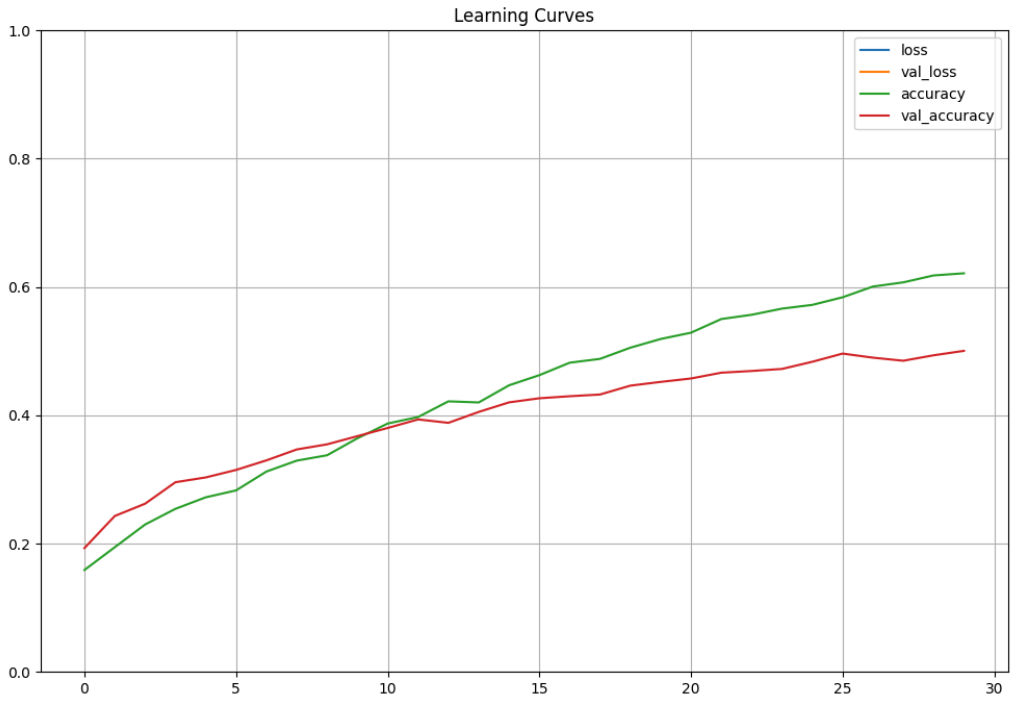


Figura (5)

Si bien el modelo 1 de VGG funciona como un buen baseline (base) de transferencia de aprendizaje, pero aún se podría elevar ese val_accuracy de 0.50 y reducir este sobreajuste. Algunas mejoras a aplicar podrían ser los Callbacks (EarlyStopping y ReduceLROnPlateau) para convencer al entrenamiento de detenerse en el pico óptimo y de afinar el learning rate automáticamente. Realizar 'Fine-tuning', con lo cual se descongelan los últimos 1–2 bloques

de VGG-16 y se los entrena con un lr muy bajo (1e-5) (esto suele dar un empujón a la validación). Asimismo, se puede emplear una estructura más ligera y/o pesada, para ello, se cambia el Dense(256) por Dense(128) o se añade un segundo Dense(128) para ver si gana capacidad sin sobreajustar. Al aplicar estas mejoras, obtuvimos un segundo modelo de vgg 'model_vgg_2'. Visualizando las curvas de aprendizaje figura (6), notamos que el modelo presenta momentos con sobreajuste y momentos que no, mejorando los resultados obtenidos al inicio. El accuracy de entrenamiento (verde) sube de forma casi lineal hasta rondar el 0.90 al final y el accuracy de validación (rojo) acompaña de cerca, alcanzando un muy buen valor de 0.85–0.86 en el conjunto de validación. Esto indica una mejor generalización y un sobreajuste mínimo o más controlado.

Otro aspecto que podemos considerar es que la buena convergencia que presenta val_loss hasta la época 22 aproximadamente, sugiere que ahí estuvo el punto de parada ideal y es un buen indicio de estabilidad. A partir de ahí el val_loss sube y val_accuracy se estanca o desciende mínimamente.

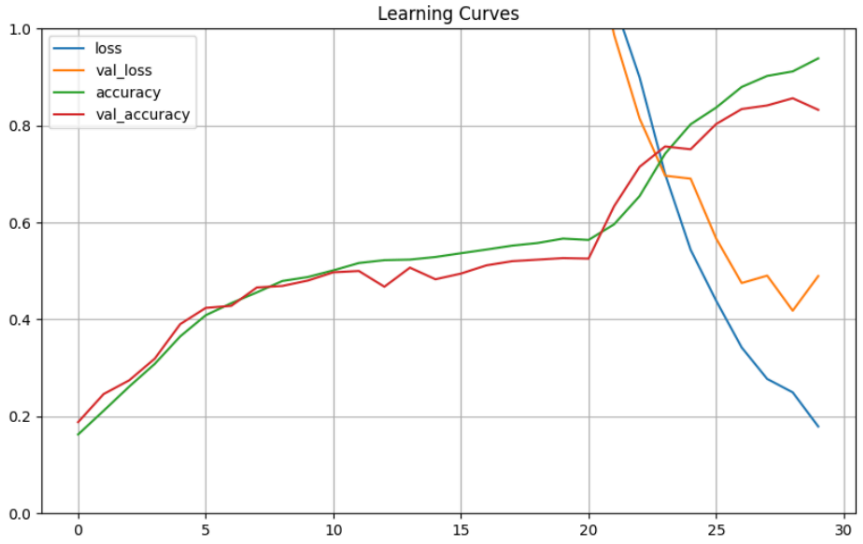


Figura (6)

Con la información rescatada del análisis podemos ver que el modelo fue mejorando en relación al inicial. Ahora se podría implementar un tercer modelo 'model_vgg_3' que

cuente con más regularización para seguir puliendo ese overfitting, aumentando el Dropout a 0.5 en Dense(128), aplicando una penalización L2 al kernel de la capa densa. Además podemos emplear menos bloques descongelados y un learning rate aún más bajo para el fine-tuning. Con estos pequeños cambios generamos las siguientes curvas figura (7):

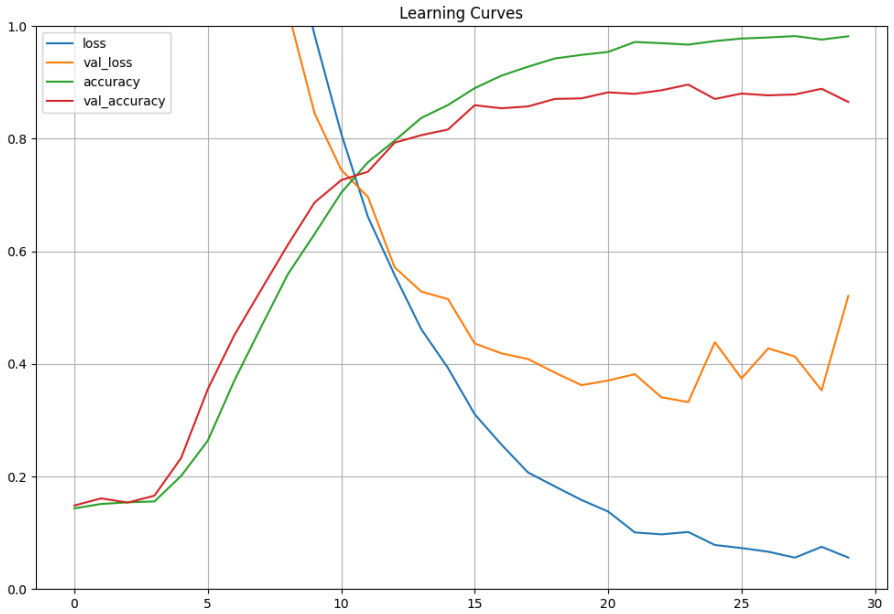


Figura (7)

Con este último ajuste, el modelo andaría bien hasta el valor de época 10, ya que la precisión de entrenamiento se mantiene ligeramente por encima de la de validación. Esto indica que el modelo está aprendiendo de manera efectiva y sus características pre-entrenadas son útiles para la tarea. Después la curva de validación (rojo) vuelve a estar por debajo de la de entrenamiento. Esta vez se obtuvieron las siguientes métricas: val accuracy: 0.55 - loss: 1.2322. Estos resultados sugieren que, a pesar de la regularización adicional, el modelo aún presenta sobreajuste o incluso podría estar cayendo en subajuste por el exceso de regularización que le impide aprender patrones más complejos para alcanzar mayor precisión.

Tomando en consideración lo mencionado en los resultados logrados para el experimento y basado en las métricas y el comportamiento de las curvas de aprendizaje, se opta por elegir al modelo ‘model_vgg_2’ para llevar a producción. Una vez que hacemos la

predicción correspondiente obtuvimos que para el caso donde la imágen es ‘feliz’, el modelo predijo ‘feliz’. Para la segunda imágen no acertó correctamente, ya que se trataba de una imágen ‘triste’ y predijo que es la clase ‘asco’. No obstante, esa última imágen presentaba rasgos de una expresión de asco, por lo que se podría considerar que el modelo no necesariamente estaría prediciendo mal. Por ende, el modelo de vgg 2 podría pulirse más optimizando sus hiperparámetros a través de una búsqueda exhaustiva o grid search.

**Resultados n°4**: En relación a los resultados obtenidos anteriormente con el modelo ‘model_vgg_2’, a partir de la búsqueda de hiperparámetros se logró alcanzar un val_accuracy máximo de 0.9229, superando el rendimiento previo del 0.85-0.86. Esta combinación óptima de hiperparámetros de ‘model_vgg_grid’ consistió en un LR=1e-05, Dropout=0.2 y Neuronas=256 en la capa dense. Esta configuración sugiere que la capa densa con mayor capacidad (256 neuronas) pudo capturar relaciones más complejas, mientras que un dropout ligeramente reducido (0.2), en conjunto con el learning rate bajo y los callbacks, permitió un entrenamiento más fino y una mejor convergencia sin caer en un sobreajuste mayor.

No obstante, tras realizar un mejor análisis de estos resultados, se determinó que, si bien se obtuvo un incremento en el valor de las métricas de validación respecto al ‘model_vgg_2’ original, esta mejora no se considera lo suficientemente significativa como para justificar el uso de esa configuración. Al comparar las curvas de aprendizaje logradas figura(8) por esta combinación del Grid Search (Val_Accuracy: 0.9229, Accuracy: 0.92) con las del model_vgg_2 original (Val_Accuracy: 0.85-0.86, Accuracy: 0.90), se observó que el comportamiento y la tendencia general de ambas curvas eran muy similares en cuanto a la forma en que el modelo aprendía y controlaba el sobreajuste.

Por ende, se optó por mantener el model_vgg_2 original como el modelo elegido para ponerlo en producción. Este modelo cuenta con 128 neuronas en la capa densa y una tasa de

Dropout de 0.3. Aunque su val_accuracy es levemente menor, estas especificaciones conllevan un menor tiempo de cómputo y una mayor robustez debido a una mayor regularización. Se considera que model_vgg_2 ofrece un equilibrio más eficiente y práctico entre un rendimiento alto y la optimización de recursos.

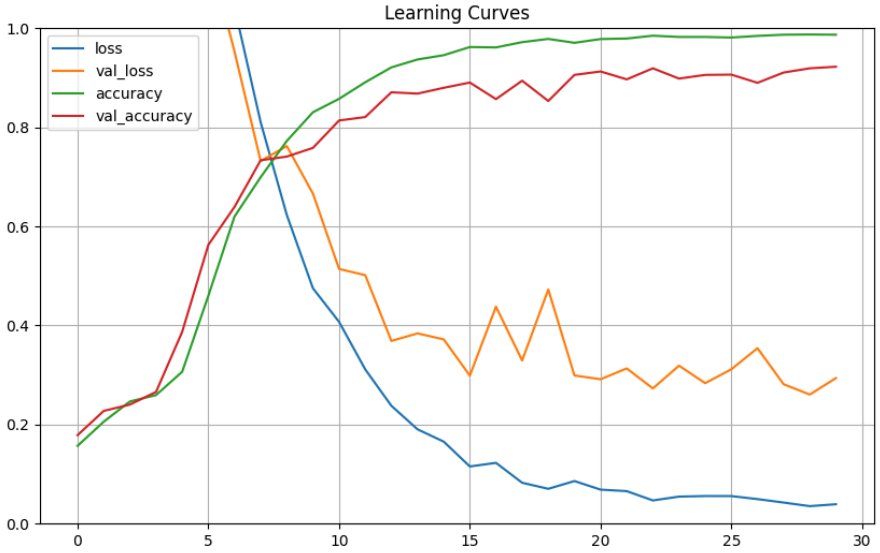


Figura (8)

# CONCLUSIÓN

El presente proyecto tuvo como objetivo general desarrollar un prototipo de reconocimiento de emociones faciales utilizando una red neuronal convolucional (CNN). Tras varios procesos de diseño, experimentación y optimización, podemos afirmar que el objetivo general ha sido cumplido dentro del plazo establecido, culminando con la creación de un modelo capaz de identificar emociones faciales con una precisión media. Para alcanzar este propósito general, se plantearon una serie de objetivos específicos que guiaron el desarrollo del proyecto y que fueron llevados a cabo: Con respecto al desarrollo de una CNN base para determinar un benchmark, se implementó una CNN simple con 2 capas convolucionales y 1 capa densa. Este 'modelo base' sirvió como punto de partida y referencia, estableciendo un benchmark inicial con un val_accuracy limitado, cercano al 0.1431. Este resultado, si bien bajo, fue crucial para comprender la complejidad de la tarea y la necesidad de arquitecturas más sofisticadas.

Considerando el desarrollo de un modelo de red neuronal convolucional basado en la arquitectura Mini-Xception: Se construyó y experimentó con un modelo basado en esa arquitectura, demostrando ser más capaz que la CNN base y alcanzando picos de validación superiores al 50%. A través de la incorporación de callbacks, se buscó estabilizar su aprendizaje, aunque los valores finales de validación se situaron ligeramente por debajo de sus picos iniciales sin estas herramientas (aproximadamente 0.59 - 0.62), priorizando la robustez y la prevención del sobreentrenamiento. Si tenemos en cuenta la optimización de los modelos en función de los hiperparámetros (Learning Rate, Épocas, Dropout y Optimizador), este aspecto fue central para mejorar el proyecto. Se realizaron diversas iteraciones de optimización, incluyendo el ajuste manual de parámetros, la aplicación de técnicas de

regularización como Dropout y Data Augmentation, y una búsqueda exhaustiva de hiperparámetros (Grid Search) sobre el modelo VGG-16.

Por otra parte, alcanzar un Accuracy superior al 55% en el conjunto de validación fue otro objetivo alcanzado. El modelo basado en Mini-Xception ya había superado este umbral. Sin embargo, el mayor logro se obtuvo al implementar el modelo VGG-16, que, tras un proceso de fine-tuning y una búsqueda de hiperparámetros, alcanzó un val_accuracy de 0.9229. Este resultado representa un avance en comparación con los modelos previos y con el objetivo propuesto.

No obstante, es importante señalar que, si bien se implementaron técnicas de regularización, como el dropout, la penalización L2, el fine-tuning con LR bajo y los callbacks de EarlyStopping y ReduceLROnPlateau, el análisis de las curvas de aprendizaje reveló una persistencia de cierto sobreajuste. Esto se evidencia entre la precisión de entrenamiento y la de validación, donde los modelos logran rendimientos altos en los datos de entrenamiento (por ejemplo para los modelos de vgg tenemos un Accuracy cercano al 98%), mientras que sus desempeños en validación se mantienen (en el 92%). Esta diferencia sugiere que el modelo, a pesar de las medidas regulatorias, tiende a ajustarse de manera muy específica a los datos que ya ha visto. En un entorno de producción real, esto podría manifestarse en una ligera disminución del rendimiento al enfrentar una diversidad de imágenes aún mayor que la del conjunto de validación. Además, el proceso de fine-tuning de los modelos VGG-16, al descongelar y entrenar una parte significativa de sus capas convolucionales, implicó un aumento considerable en el tiempo de entrenamiento. Este factor es una consideración importante para futuros desarrollos, donde la eficiencia computacional puede ser crucial.

En conclusión, este proyecto ha logrado desarrollar un prototipo de reconocimiento de emociones faciales con buena precisión, sentando bases para futuras investigaciones y

aplicaciones en el campo del apoyo a niños con Trastorno del Espectro Autista. Los objetivos propuestos han sido alcanzados, y los desafíos identificados abren paso para la mejora continua del sistema.

**REFERENCIAS**

Lorenzo-Lledó, G., Lorenzo-Lledó, A., & Rodríguez-Quevedo, Á. (2024). Análisis mediante inteligencia artificial de las emociones del alumnado autista en la interacción social con el robot NAO. *RED. Revista de Educación a Distancia, 78*(24). DOI: 10.6018/red.588091

Rodríguez, A. M. (2021). "EXPRESIÓN EMOCIONAL EN PERSONAS CON TRASTORNO DE ESPECTRO AUTISTA. PROPUESTA DE INTERVENCIÓN, DE CASO ÚNICO, CENTRADA EN LA FAMILIA." https://uvadoc.uva.es/bitstream/handle/10324/49271/TFG-G5004.pdf

Salhi, I., Qbadou, M., Gouraguine, S., Mansouri, K., Lytridis, C., & Kaburlasos, V. (2022). Towards Robot-Assisted Therapy for Children With Autism—The Ontological Knowledge Models and Reinforcement Learning-Based Algorithms. Frontiers in Robotics and AI, 9. https://doi.org/10.3389/frobt.2022.713964

Holeva V, Nikopoulou VA, Lytridis C, Bazinas C, Kechayas P, Sidiropoulos G, Papadopoulou M, Kerasidou MD, Karatsioras C, Geronikola N, Papakostas GA, Kaburlasos VG, Evangeliou A. Effectiveness of a Robot-Assisted Psychological Intervention for Children with Autism Spectrum Disorder. J Autism Dev Disord. 2024 Feb;54(2):577-593. doi: 10.1007/s10803-022-05796-5. Epub 2022 Nov 4. PMID: 36331688; PMCID: PMC9638397. https://doi.org/10.1007/s10803-022-05796-5

Conti, D., Di Nuovo, S., Buono, S., Trubia, G., & Di Nuovo, A. (2015). Use of robotics to stimulate imitation in children with Autism Spectrum Disorder: A pilot study in a clinical setting. 2015 24th IEEE International Symposium on Robot and Human Interactive Communication (RO-MAN). https://doi.org/10.1109/roman.2015.7333589

Recuperado de: chrome-extension://efaidnbmnnnibpcajpcglclefindmkaj/https://autismonavarra.com/wp-content/uploads/2012/05/03-Ppios-y-estrategias.pdf

Gándara Rossi, CC (2007). Implementar principios de intervención educativa y estrategias de comunicación de TEACCH para personas con autismo. Revista de Logopedia, Foniatría Y Audiología , 27 (4), 173–185. https://doi.org/10.1016/s0214-4603(07)70086-4

Fatima, S. A., Kumar, A., & Raoof, S. S. (2021). Real Time Emotion Detection of Humans Using Mini-Xception Algorithm. IOP Conference Series: Materials Science and Engineering, 1042(1), 012027. https://doi.org/10.1088/1757-899x/1042/1/012027

Alzubaidi, L., Zhang, J., Humaidi, A. J., Al-Dujaili, A., Duan, Y., Al-Shamma, O., Santamaría, J., Fadhel, M. A., Al-Amidie, M., & Farhan, L. (2021). Review of deep learning: concepts, CNN architectures, challenges, applications, future directions. Journal of Big Data, 8(1). https://doi.org/10.1186/s40537-021-00444-8

Fatima, S. A., Kumar, A., & Raoof, S. S. (2021). Real Time Emotion Detection of Humans Using Mini-Xception Algorithm. IOP Conference Series: Materials Science and Engineering, 1042(1), 012027. https://doi.org/10.1088/1757-899x/1042/1/012027

Zhao, X., Wang, L., Zhang, Y., Han, X., Deveci, M., & Parmar, M. (2024). A review of convolutional neural networks in computer vision. Artificial Intelligence Review, 57(4). https://doi.org/10.1007/s10462-024-10721-6